Alma Mater Studiorum · Università di Bologna

Second Cycle Degree
Artificial Intelligence

Combinatorial Decision Making and Optimization

Student:
Matteo Canghiari

Academic Year 2025/2026

Chapter 1

Introduction

Chapter 2

Modelling in CP

We need to start from the **model**: it doesn't just give us a formal representation of our problem, but also the modelling choices directly impact the search performance.

2.1 Formalization as a Constraint Satisfaction Problem (CSP)

A formal definition of any **Constraint Satisfaction Problem** (*CSP*) can be described as follows.

Definition

A CSP is a triple $\langle \mathbf{X}, \mathbf{D}, \mathbf{C} \rangle$ where:

- **X** is a set of decision variables $\{X_1, \ldots, X_n\}$, which we have to associate a value.
- **D** is a set of domains $\{D_1, \ldots, D_n\}$, where D_i is the domain of the decision variable X_i with $i \in \{1, \ldots, n\}$.
- **C** is a set of constraints $\{C_1, \ldots, C_m\}$. Every constraint C_i with $i \in \{1, \ldots, m\}$ is a mathematical relation over a subset of variables $\{X_j, \ldots, X_k\}$, denoted as $C_i\{X_j, \ldots, X_k\}$. Being a relation means a subset of the cartesian product of all the domains of the decision variables.

Any CSP would be described by this triple $\langle X, D, C \rangle$. A **solution** of a Constraint Satisfaction Problem is an assignment of values to the decision variables which satisfies all the constraints at the same time.

2.2 Constraint Optimization Problem

Previously, we talked about **satisfaction**: since we have constraints, the solution must satisfy all of them simultaneously. Anyway, sometimes we need to deal with **optimization**, that it's the same thing as before but there is an additional term, named **objective function**.

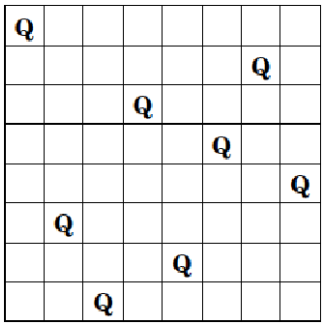
In this case, the formal definition of a **Constraint Optimization Problem** is described also by f, the formalization of the optimization criterion as an objective function. Therefore, our original triple becomes $\langle$**X,D,C,f**$\rangle$, where:

- **X** set of decision variables.
- **D** set of domains of the decision variables.
- **C** set of constraints applied to the decision variables.
- **f** formalization of the optimization criterion. It can be a **minimization objective** (*minimize* f) or a **maximization objective** (*maximize* $-f$).

Let's take a look at the following problems, understanding what decision variables, domains and constraints are really.

N-Queens

e.g. Place n queens in an $n \times n$ board so that no two queens can attack each other.



There are many ways to model this problem, however we try to solve it using the obvious approach. Our goal is to place n queens in a such way there is only one of them for each row, column and diagonal. We begin by defining the decision variables and their domains.

Decision variables and domains.

- Since each row must contain exactly only one queen, we define n decision variables $[X_1, X_2, \ldots, X_n]$; mapping one variable at each row we can satisfy the *no row attack* condition.

- The domain values $\{1, \ldots, n\}$ represent the available columns. An assignment $X_i = j$ indicates that queen in row i is positionated in column j.

To ensure no two queens attack each other, we must address three types of constraints:

1. *Row attacks.*
2. *Column attacks.*
3. *Diagonal attacks.*

Our modelling choice about decision variables allows us to satisfy the row constraint by design. Because each X_i represents a distinct row, we only need to explicitly define constraints for the remaining two.

Constraints.

- **alldifferent**$([X_1, X_2, \ldots, X_n])$.
- for all $i < j$ $|X_i - X_j| \neq |i - j|$.

These constraints prevent column and diagonal attacks, respectively. Specifically, the second constraint ensures that no two queens occupy the same diagonal, which occurs when the distance between their colums is equal to the distance between their rows.

Sudoku

e.g. Sudoku.

	6		1		4		5	
		8	3		5	6		
2								1
8			4		7			6
		6				3		
7			9		1			4
5								2
		7	2		6	9		
	4		5		8		7	

Solving the famous sudoku problem, we can do something similar to the previous example. Even in this problem, we get some decision making processes subject to constraints. As before, we begin by defining the decision variables.

Decision variables.

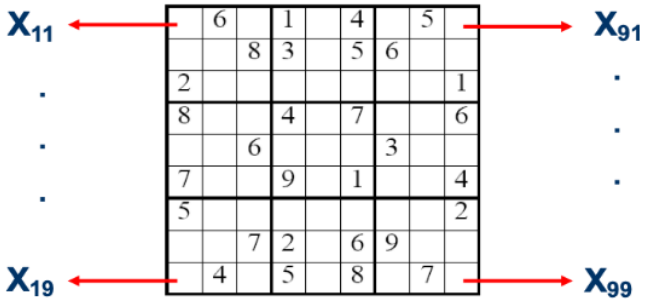
We start modelling the decision variables, which are in this case the **empty cells**: for every cell we have to give a decision. Anyway, focusing only on the empty cells we cannot model properly the given problem; also, the already

assigned cells can somehow influence our decisions.

So, what do we do?
Simply, in our model these variables will be assigned to the current values shown in the image.

Domains.
The domain values are any value in the set $\{1, \ldots, 9\}$.

Summing up, we can create 9×9 different decision variables, where each of them has a domain $\{1, \ldots, 9\}$; except for those variables that are already associated to a value.



Constraints.
- **alldifferent**$([X_{11}, X_{21}, X_{31}, \ldots, X_{91}])$.
- **alldifferent**$([X_{11}, X_{12}, X_{13}, \ldots, X_{19}])$.
- **alldifferent**$([X_{11}, X_{21}, X_{31}, X_{12}, X_{22}, X_{32}, X_{13}, X_{23}, X_{33}])$.
- Initial assignments, already described in the image (*for instance,* $X_{21} = 6$).

The first three constraints ensure different values for any row, column and box of the sudoku table.

Map colouring

e.g. Optimal map colouring.

Our goal is to colour regions such that neighboring regions are assigned different colors, using the minimum number of colors.

Decision variables and domains.
Every decision variable would be a region and any single domain value represents a different color. More formally, X_i with $i \in \{1, \dots, n\}$ are the decision variables, while $\{1, \dots, n\}$ it's the domain.

Since we have n regions, we could use n different colors satisfying the map colouring problem. However, we want to find out the minimum number of colors that we can use.

Constraints.
- $X_i \neq X_j$ for each neighboring region i and j.

We specify inside the constraint the necessary condition for stating when a region is a neighbor of an other one, since not all the regions are consecutive between each other.

Objective variable.
- $f = max(X_i)$
  where:
  - $X_i \rightarrow$ number of colors used by the map colouring
  - $max(\dots) \rightarrow$ function used to detect how many colors were used

Objective.
In order to obtain the minimum number of colors for the map colouring problem we have to minimize the objective function f.

2.3 Task scheduling

Constraint Programming is really powerful when we have to deal within **scheduling problems**. Typically, such problems are described by the following definition.

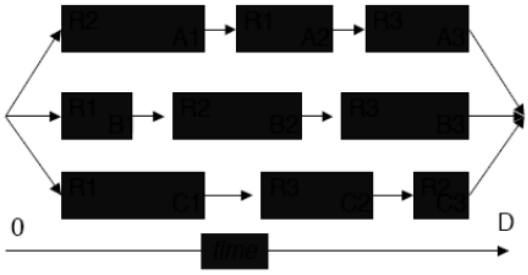
Task scheduling definition

Schedule n tasks on a machine, in time D, by obeying the temporal and precedence constraints:
1. Each task t_i has a specific fixed processing time p_i.
2. Each task t_i can be started after its release date r_i and must be completed before its deadline d_i.
3. Tasks cannot overlap in time.
4. Precedence relations must be respected.

Example

e.g. Task scheduling general problem.

Before we get into the solution, let's take a look at the image below.



Observing the figure, we can state:

- Every black box is a task t_i and its size determines the fixed processing time p_i.
- Every flow does not overlap tasks in the same row.
- Every flow respects the precedence relationships.

We could express decision variables, domains and constraints of any kind of task scheduling problem following a generic approach.

Generic approach.
Since the more interisting part of the optimization problem is the starting time of tasks t_i, the same decision variables can be directly described as the starting time; this is possible since we already know how much time any process requires.

Decision variables and domains.
We call $Start_i$ the decision variables representing the starting time of tasks t_i, where each of them takes a value from $[0 \dots D]$.

The domain $[0 \dots D]$ defines all the time we have available to compute our tasks t_i.

Constraints.

- Respect of release date and deadline.
  - for all $i \in \{1, \dots, n\}$ such that $r_i \leq Start_i \leq d_i - p_i$
    where:
    * $r_i \rightarrow$ release time
    * $d_i \rightarrow$ deadline time
    * $p_i \rightarrow$ processing time
    * $Start_i \rightarrow$ starting time of the task t_i

- No overlap in time.
  - for all $i < j \in \{1, \ldots, n\}$ such that $(Start_i + p_i \leq Start_j) \vee (Start_j + p_j \leq Start_i)$
    where:
    * $p_i \rightarrow$ processing time of the task t_i
    * $p_j \rightarrow$ processing time of the task t_j
    * $Start_i \rightarrow$ starting time of the task t_i
    * $Start_j \rightarrow$ starting time of the task t_j
- Precedence constraints.
  - $Start_i + p_i \leq Start_j$ for each pair of tasks $t_i \mapsto t_j$
    where:
    * $p_i \rightarrow$ processing time of the task t_i
    * $Start_i \rightarrow$ starting time of the task t_i
    * $Start_j \rightarrow$ starting time of the task t_j
    * $t_i \mapsto t_j \rightarrow$ precedence relation between tasks t_i and t_j

Final mark.
Before moving on, let's take a look at the final two constraints.

2nd No overlap in time.
The starting and ending time of any pair of tasks (t_i, t_j) must be either the same time or ordered time, but they are never allowed to overlap each other.

3rd Precedence constraints.
In almost of the cases, the second part of the previous constraint doesn't even exist ($\vee$ $(Start_j + p_j \leq Start_i)$), since in general the output of a task it's used by another task as input, relying on the precedence relationships.

2.4 Constraints

Constraints fall into two broad categories:

1. Most of them rely on declarative relations among objects. They describe an **intensional representation**, since we show the mathematical relationship in a **declarative** way of the constraint (*e.g.* $X > Y$).
2. Any constraint can be expressed by listing all the allowed combinations. This set of constraints follows exactly the mathematical relationship definition: any constraint can be seen as a subset of the cartesian product of all the domains. Generally, these constraints are named **extensional representation** or **table constraints** (*e.g.* $C(X_1, X_2) = \{(0,0), (0,2), (1,3,(2,1))\}$).

Additionally, constraints share the same features, in particular:

- **Order does not matter**.
$$X + Y \leq Z \text{ is the same as } Z \geq X + Y$$
- **Non-directional**.
A constraint between decision variables X and Y can be used to infer domain information on Y given domain information on X and vice versa.
- **Rarely independent**.
Shared variables as communication mechanism between constraints. For instance, using the following relations $C_1 \rightarrow C_2 \rightarrow C_3$, we can exploit C_2 to infer something between C_1 and C_3.

Intensional constraints examples

e.i. There are several constraints regarding intensional representation category.

- **Algebraic expressions**.
  - $X_1 > X_2$
  - $X_1 + X_2 = X_3$
- **Extensional constraints**.
$$(X, Y, Z) \text{ in } \{(1, 3, 5), (2, 4, 6), (3, 5, 4)\}$$
- **Logical relations**.
$$(X < Y) \vee (Y < Z) \rightarrow C$$
- **Global constraints**.
  - $\mathit{alldifferent}([X_1, X_2, X_3])$ instead of:
    $X_1 \neq X_2, X_1 \neq X_3, X_2 \neq X_3$
  - $\mathit{noOverlap}([\mathit{Start}_i, \ldots, \mathit{Start}_n], [p_1, \ldots, p_n])$ instead of:
    for all $i < j \in \{1, \ldots, n\}, (\mathit{Start}_i + p_i \leq \mathit{Start}_j) \vee (\mathit{Start}_j + p_j \leq \mathit{Start}_i)$

  Rule of thumb: if we have a relationship between all the decision variables, we always choose the **global constraint** instead of the hand-made one.
- **Meta-constraints**.
  A meta-constraint is a constraint inside an other constraint.
$$\sum_i (X_i > t_i) \leq 5$$
  where:
  - $X_i \rightarrow$ decision variable
  - $t_i \rightarrow$ parameter

  We are simply counting how many decision variables satisfy the actual

constraint. Roughly speaking, it's created a hidden boolean variable and any time $X_i > t_i$ the variable is set to 1. We merge these boolean variables swearing that the decision variables are at most 5.

2.5 Modelling is critical!

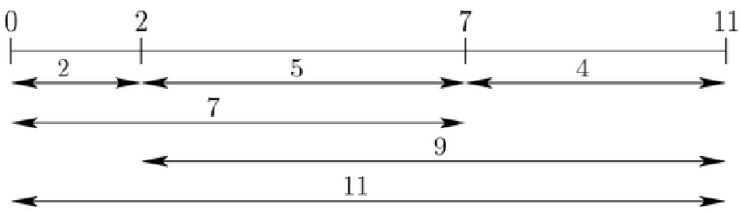
Every choice done can affect our model, like:

- **Choice of decision variables and their domains**.
  The search space is defined as the product of all domains for each decision variable, in particular:
  $$|D(X_1)| \times |D(X_2)| \times \cdots \times |D(X_n)|$$
  Therefore, the search space could be **exponential** in size.
- **Choice of constraints**.
  Selecting the ideal constraints might be the essential key to solve our optimization problem. Propagating constraints the size of the search space can be reduced.

2.6 Golomb ruler

Golomb ruler problem

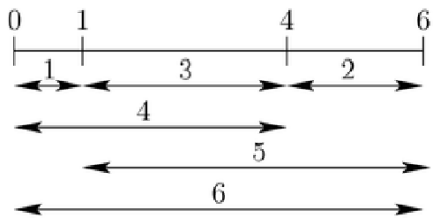
e.g. Starting from an empty ruler we have to place m marks.



At first sight, the problem may appear easy to solve, but instead it's really difficult. Our goal is to place m different marks on a ruler such that the following conditions are satisfied:

1. Distance between each pair of marks is different.
2. The length of the ruler obtained is minimum.

As shown in the image above, we placed *four marks* over the ruler, achieving a total length equal to 11, even though this is not the optimal solution.



With this second approach, we reach a solution up to 6 always placing *four different marks*.

Decision variables and domains.
It's quite easy to determine the given marks as m different decision variables, but what makes the Golomb ruler problem so difficult is the choice of their domains, in particular:

- $[X_1, X_2, \ldots, X_m]$ are our decision variables.
- $\{0, 1, \ldots, 2^{m-1}\}$ is the domain of each decision variable. Often the upper limit coincides within 2^{m-1}, since also in the worst case we are able to place m marks using the powers of 2, guaranteeing at least one solution.

Constraints.

- for all $i_1 < j_1, i_2 < j_2, i_1 \neq i_2$ or $j_1 \neq j_2$ $|X_{i1} - X_{j1}| \neq |X_{i2} - X_{j2}|$

Making sure that all pairs of distances have a different difference.

Objective.
Since we don't know which decision variable will have the highest value, we have to minimize the max value (*the max must be minimum as much as possible*).

$$minimize(max([X_1, X_2, \ldots, X_m]))$$

Final mark.
It's not a good model, since we have absolute values in both sides of the constraint, making hard the propagation. It would be nice to delete these absolute values to get a smoother model. We need to remodel.

Sometimes we introduce **additional variables** in our model, even though we said previously more variables we have more large would be the search space. However, adding variables in order to obtain a bigger search space helps us to express constraints much better than before and, also, spread their propagations.

Better model

e.g. Starting from an empty ruler we have to place m marks.

Auxiliary variables.

In the Golomb ruler problem we can introduce a new additional variable D_{ij} that represents the distance between mark i and mark j.

Constraints.
The new constraints should be as follows:
- for all $i < j,\ D_{ij} = |X_i - X_j|$
- *alldifferent*$([D_{12}, D_{13}, \ldots, D_{(m-1)m}])$

Thanks to the additional variable D_{ij} we can directly express a global constraint.
- for all $i < j,\ D_{ij} = |X_i - X_j|$
- *alldifferent*$([X_1, X_2, \ldots, X_m])$
- *alldifferent*$([D_{12}, D_{13}, \ldots, D_{(m-1)m}])$

The second constraint introduced is named **implied constraint**, as the same name suggests, this is a constraint **logically redundant** but **computationally significant**. *alldifferent*$([X_1, X_2, \ldots, X_m])$ is not a new constraint, since different distances (*the third constraint*) already implies that also decision variables $X_1, \ldots, X_m$ must be different.

2.7 Symmetry in CSPs

The model designed could have **symmetry**: a state leading to a solution or a failure that have many symmetrically equivalent states. Symmetry is divided into:

- **Variable symmetry**.
  A **permutation** π of the **variable indeces** such that for each feasible or unfeasible assignment, we can rearrange the variables according to π and obtain another feasible or unfeasible assignment.

  Variable symmetry in Golomb ruler problem

  e.g. Permuting variable assignments.

  $$\langle X_1 = 0, X_2 = 1, X_3 = 4, X_4 = 6 \rangle$$

  Starting from the above solution, we can define different ways to assign values or marks over the ruler, obtaining always the same conclusion. For any solution or failure composed by m entities, we can derive $m!$ permutations. For instance, looking at the previous example we get $4 \times 3 \times 2 = 24$ same conclusions.

- **Value symmetry**.

A **permutation** π of the **values** such that for each feasible or unfeasible assignment, we can rearrange the values according to π and obtain another feasible or unfeasible assignment.

Value symmetry in Golomb ruler problem

e.g. Values can be permuted as follows:

$$\langle 0 \to 0, 1 \to 2, 2 \to 1, 3 \to 3, 4 \to 5, 5 \to 4, 6 \to 6 \rangle$$

Even for value permutations we get the same factorial complexity, $m!$ where m is the number of values assigned.

2.8 Symmetry breaking constraints

We need a way to reduce the set of solutions and consequently the huge search space from the symmetries. A general approach is to impose an **ordering** between the decision variables to avoid the permutations, in particular:

$$X_1 \leq X_2 \leq \cdots \leq X_n \text{ when } [X_1, X_2, \ldots, X_n] \text{ are all symmetric.}$$

Notice, at least one solution from each set of symmetrically equivalent solutions must remain.

Improved model

Symmetry breaking constraints.
- $X_1 < X_2 < \cdots < X_m$
- $X_1 = 0$
- $D_{12} < D_{(m-1)m}$

Notice: symmetry breaking constraints and implied constraints are not the same thing. An implied constraint is a logical implication that does not change the solution at all, while a symmetry breaking constraint is not a logical implication, having it inside our model could change the state of the solution.

New objective.
- $minimize(X_m)$

At first sight, any global constraint seems to be a good constraint for our solver, like *alldifferent*(...) constraint. However, after we introduced these symmetry breaking

constraints, the *alldifferent*(...) constraint is not useful anymore, it becomes **semantically** and **computationally redundant**. Symmetry breaking constraints enable **constraint semplification**.

Simplified improved model

Symmetry breaking constraints.

- $X_1 < X_2 < \cdots < X_m$
  alldifferent$([X_1, X_2, \ldots, X_m])$ constraint becomes redundant. In the process of remodelling, delete *alldifferent*$([X_1, X_2, \ldots, X_m])$ constraint can be a good idea to avoid redundancy. In addition, also the first constraint introduced in the previous section (*for all* $i < j,\ D_{ij} = |X_i - X_j|$) can be simplified as follows:
  - for all $i < j,\ D_{ij} = X_i - X_j$

  We don't need anymore absolute values, exploiting the decision variables order imposed by symmetry breaking constraints.

2.9 Dual viewpoint

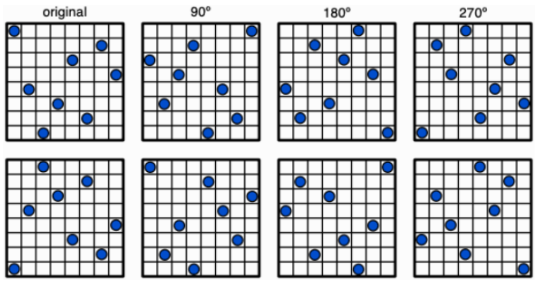
Modelling phase is critical. Starting from a new problem and observing it by different **perspectives** we can design several models. Generally, every model exhibits a different representation of the original problem, in particular any of them have:

1. Different decision variables.
2. Different domains.
3. Different constraints.
4. Different size of the search space.

Which model is the best one?

Symmetries of n-queens problem

e.g. Solution of n-queens problem.



Just rotating and flipping the board we end up with *seven* additional symmetric solutions of the top-left original one. Additionally, we cannot impose an ordering constraint as we did before for the ruler problem. We need a new **viewpoint** to avoid these *seven* symmetric feasible solutions.

Decision variables and domains.
We represent the $n \times n$ board as a boolean board and every decision variable is a boolean variable like $B_{ij} \in \{0, 1\}$.

Attacking constraints.
- $\sum B_{ij} = 1$ on all rows and columns.
- $\sum B_{ij} \leq 1$ on all the diagonals.

Now we are able to convert geometric symmetry to boolean symmetry.

Symmetry breaking constraints.
We determine the following list of steps to avoid symmetric solutions.

1. Flat the 2-D matrix to a single sequence of variables. Simply, we take each row of the board and we append it at the end of the sequence.
2. Every symmetric configuration corresponds to a **variable permutation** of the original solution, which is easy to define.
3. We then impose an order between a solution and all the solutions obtained by the *seven* permutations:

$$lex \leq (B, \pi(B)) \text{ for all } \pi$$

where:
- $B \rightarrow$ flatten version of the 2-D matrix
- $\pi(B) \rightarrow$ symmetries of the original solution

Chapter 3

Local consistency, constraint propagation and global constraints

...